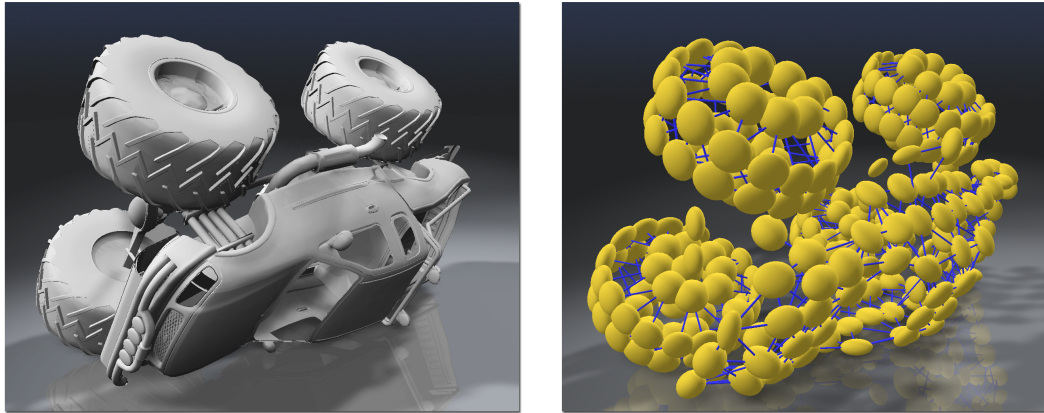


# Solid Simulation with Oriented Particles

Matthias Müller

Nuttapong Chentanez

NVIDIA PhysX Research



**Figure 1:** Using particles with orientation enables us to simulate a complex model like this monster truck with plastically deforming body, free spinning wheels with soft tires, and high fidelity mesh skinning in real time all with a sparse physical representation.

## Abstract

We propose a new fast and robust method to simulate various types of solid including rigid, plastic and soft bodies as well as one, two and three dimensional structures such as ropes, cloth and volumetric objects. The underlying idea is to use oriented particles that store rotation and spin, along with the usual linear attributes, i.e. position and velocity. This additional information adds substantially to traditional particle methods. First, particles can be represented by anisotropic shapes such as ellipsoids, which approximate surfaces more accurately than spheres. Second, shape matching becomes robust for sparse structures such as chains of particles or even single particles because the undefined degrees of freedom are captured in the rotational states of the particles. Third, the full transformation stored in the particles, including translation and rotation, can be used for robust skinning of graphical meshes and for transforming plastic deformations back into the rest state.

**CR Categories:** I.3.5 [Computer Graphics]: Computational Geometry and Object Modeling—Physically Based Modeling; I.3.7 [Computer Graphics]: Three-Dimensional Graphics and Realism—Animation and Virtual Reality

**Keywords:** oriented particles, shape matching, position based dynamics

**Links:**  [DL](#)  [PDF](#)

### ACM Reference Format

Müller, M., Chentanez, N. 2011. Solid Simulation with Oriented Particles. *ACM Trans. Graph.* 30, 4, Article 92 (July 2011), 9 pages. DOI = 10.1145/1964921.1964987 <http://doi.acm.org/10.1145/1964921.1964987>.

### Copyright Notice

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or direct commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701, fax +1 (212) 869-0481, or [permissions@acm.org](mailto:permissions@acm.org).  
© 2011 ACM 0730-0301/2011/07-ART92 \$10.00 DOI 10.1145/1964921.1964987  
<http://doi.acm.org/10.1145/1964921.1964987>

## 1 Introduction

Physical simulation of solids has been investigated for more than two decades in computer graphics. In contrast to the computational sciences, computer graphics is more concerned with creating the overall look and feel of objects than the accurate reproduction of their small scale behavior. Also, artists require easy tuning of the physical attributes as well as full control of object behavior.

Lately, the trend in solid simulation in computer graphics has been to increase the accuracy of the mathematical models. This typically requires an increase in their complexity. Advantages of using representations based on continuum mechanics are that object behavior can be controlled using physical parameters such as Young's modulus, and that the discretization converges toward the continuous solution with increasing mesh resolution.

However, in computer games, where robustness and speed are often more essential than accuracy, simpler unconditionally stable geometric methods such as position based dynamics (PBD) [Müller et al. 2006] can be sufficient to create the desired physical effects. For these reasons we decided to come up with a method that is as simple and as fast as possible, yet able to create the desired visual fidelity required in many computer graphics applications. Our method is based on generalizations of PBD and the shape matching approach [Müller et al. 2005]. The novel idea of using oriented particles in connection with shape matching allows us to create complex dynamic objects with only a small number of simulation particles. This makes turning a visual mesh into a physical object a simple task which can be performed in just a few minutes.

In the first part of the paper we will present our research contributions which are

- An extension of PBD to handle orientation and angular velocity of particles
- A generalized formulation of the shape matching method incorporating particle orientations. This new formulation guarantees stability for arbitrary numbers and arrangements of particles.

- To leverage the orientation information to (1) approximate shapes by ellipsoids instead of spheres for more accurate collision handling and (2) to skin a visual mesh to the simulation nodes.

In the second part we will describe our content creation and simulation framework, which we developed based on our the new method, along with a variety of scenes demonstrating the versatility of the system.

## 2 Related Work

Formulating a unified solver for various solid material types has been an active topic in computer graphics in recent years. As a unified model, [Stam 2009] represents solids with simplices of various dimensions. [O'Brien et al. 1997], [Jansson and Vergeest 2003] and [Lenoir and Fonteneau 2004] describe ways to couple deformable and rigid bodies. [Sifakis et al. 2007] use soft and hard binding to transmit forces between different representations. Point-based approaches lend themselves especially well to unified solid simulation. One such method based on continuum mechanics was proposed by [Müller et al. 2004] and extended later by [Pauly et al. 2005; Gerszewski et al. 2009]. These approaches use moving least squares (MLS) to derive a deformation field from particle positions. MLS is only stable if local particle neighborhoods are in non-degenerate configurations. This problem was fixed by Martin et al. [2010] who introduced the concept of *elastons*. In addition to the deformation field, the elaston approach also stores derivatives. This way, particles in zero, one, two and three-dimensional configurations can be simulated robustly using generalized MLS (GMLS). Their approach was not designed for real-time use.

Our goal was to apply a similar idea to the geometric approach of shape matching [Müller et al. 2005]. In an analogous way, basic shape matching fails when particles are arranged in singular patterns. We fix this by storing orientation information on the particles. The link to the elaston idea and GMLS is that orientations can be viewed as derivative information of a normalized deformation field. An oriented particle is basically the simplest piece of information to fix the singularity problem. Due to its simplicity, the oriented particle approach is significantly faster than using elastons.

[Becker et al. 2009] use particles in connection with SPH-based forces to simulate various solids. They solve the singularity problem of degenerate cases by stabilizing the polar decomposition as in [Schmedding and Teschner 2008]. The missing rotation information is completed statically and not simulated as in our case. In the extreme case of a single particle, for instance, stabilized polar decomposition always returns the identity matrix for the particle's orientation, while in our case the angular quantities evolve physically in time. Another problem arises when exactly two Eigenvalues of the moment matrix are zero. This happens permanently for one-dimensional structures with stretched rest state. While one missing Eigenvector can be determined uniquely using the cross product of the other two, the directions of two missing Eigenvectors are not unique. To avoid jittering, they need to be chosen consistently over time which is only possible with a state variable. This is exactly what our orientation information on the particles provides.

The problems of simulating one, two and three-dimensional solids have been studied independently as well. To mention only a few papers, [Pai 2002] simulates elastic rods such as hair, wire and threads using the Cosserat theory by formulating a Boundary Value Problem (BVP) which does not have a guaranteed running time bound in general. [Bertails et al. 2006] and [Spillmann and Teschner 2007; Bertails 2009] reduced the complexity of the approach to be quadratic and linear in time, respectively. An approach based on discrete differential geometry was proposed by [Bergou et al.

2010] to allow for simulating thin viscous liquid threads like dripping honey.

The simulation of thin deformable bodies has been an active research field as well. [Provot 1995] were among the first to use mass-spring networks for simulating cloth. Later [Baraff and Witkin 1998] and [Bridson et al. 2003] proposed to use semi-implicit integration to increase stability and allow for larger time steps. [Volino et al. 2009] used non-linear springs to capture cloth behavior more faithfully. Various authors have addressed the problem of cloth stretchiness using the idea of strain limiting such as [Goldenthal et al. 2007]. Beside cloth simulation [Grinspun et al. 2003] and many others investigated the more general problem of thin shell simulation.

In computer graphics one of the most popular approach to simulating volumetric solids is the Finite Element Method (FEM) with linear tetrahedral elements. This technique was used in [O'Brien and Hodgins 1999] to simulate fracture and by [O'Brien et al. 2002; Bargteil et al. 2007; Wojtan and Turk 2008] to model plastic materials. The co-rotational formulation was introduced by [Müller and Gross 2004] to reduce visual artifacts in connection with linear elements and large deformations. To reduce the number of elements required for simulation Martin et al. [2008] generalized the traditional method to include general polyhedral elements.

The idea of attaching orientation information to particles and the term "oriented particle" was introduced by [Szeliski and Tonnesen 1992]. They used the additional information to define special energy potentials that let non-connected particles form surfaces rather than volumetric objects. Their approach is quite different from ours since they do not use a mesh, their particles are isotropic, they do not handle collision detection nor use the particles for skinning a visual mesh.

## 3 Generalized Shape Matching

As mentioned previously, the main idea behind our approach is to use particles that have both position and orientation. We will first show how the shape matching approach [Müller et al. 2005] benefits from this additional piece of information because shape matching lies at the core of our method.

Given  $n$  particles with rest position  $\bar{\mathbf{x}}_i$ , current position  $\mathbf{x}_i$  and mass  $m_i$  we are looking for a global translation  $\mathbf{t}$  and rotation  $\mathbf{R}$  of the rest state that matches the current positions optimally in a least squares sense. For this the moment matrix  $\mathbf{A}$  has to be computed as

$$\mathbf{A} = \sum_i m_i (\mathbf{x}_i - \mathbf{c})(\bar{\mathbf{x}}_i - \bar{\mathbf{c}})^T \in \mathbb{R}^{3 \times 3}, \quad (1)$$

where the mass centers are defined as

$$\mathbf{c} = \sum_i m_i \mathbf{x}_i / \sum_i m_i \quad \text{and} \quad (2)$$

$$\bar{\mathbf{c}} = \sum_i m_i \bar{\mathbf{x}}_i / \sum_i m_i. \quad (3)$$

The polar decomposition  $\mathbf{A} = \mathbf{R}\mathbf{S}$  yields the least squares optimal rotation  $\mathbf{R}$  of the original shape into the actual configuration while for the translation we have  $\mathbf{t} = \mathbf{c} - \bar{\mathbf{c}}$ . Using this transformation, the goal position of each particle becomes

$$\mathbf{g}_i = \mathbf{R}(\bar{\mathbf{x}}_i - \bar{\mathbf{c}}) + \mathbf{c}. \quad (4)$$

If the particles are close to co-planar or co-linear,  $\mathbf{A}$  becomes ill-conditioned or even singular. In this case, the optimal rotation  $\mathbf{R}$  is not well defined, causing the simulation to be unstable. This

problem can be fixed by using the orientation information of the particles.

Let us assume we have two groups of particles with their respective moment matrices  $\mathbf{A}_1$  and  $\mathbf{A}_2$ . To compute the total moment matrix  $\mathbf{A}$  of the union of the particles, the two matrices cannot simply be added because each one is computed w.r.t. its own center of mass. Fortunately there is an easy way to fix this problem. In their paper, [Rivers and James 2007] reformulated Eq. (1) for fast summation into the following form:

$$\mathbf{A} = \sum_i m_i \mathbf{x}_i \bar{\mathbf{x}}_i^T - M \mathbf{c} \bar{\mathbf{c}}^T, \quad (5)$$

where  $M = \sum_i m_i$ . Now let us assume we could define a moment matrix  $\mathbf{A}_i$  for a single particle w.r.t. its center of mass  $\mathbf{x}_i$ . Using Eq. (5) we shift the center of mass of each  $\mathbf{A}_i$  to the global one as

$$\mathbf{A} = \sum_i \left( \mathbf{A}_i + m_i \mathbf{x}_i \bar{\mathbf{x}}_i^T - m_i \mathbf{c} \bar{\mathbf{c}}^T \right) \quad (6)$$

and Eq. (1) generalizes to

$$\mathbf{A} = \sum_i \left( \mathbf{A}_i + m_i \mathbf{x}_i \bar{\mathbf{x}}_i^T \right) - M \mathbf{c} \bar{\mathbf{c}}^T \quad (7)$$

To compute the moment matrix of a particle with orthonormal orientation matrix  $\mathbf{R}$  we integrate Eq. (1) over the particle's volume yielding

$$\begin{aligned} \mathbf{A}_{\text{sphere}} &= \int_{V_r} \rho(\mathbf{R}\mathbf{x}) \mathbf{x}^T dV = \rho \mathbf{R} \int_{V_r} \mathbf{x} \mathbf{x}^T dV \\ &= \frac{4}{15} \pi r^5 \rho \mathbf{R} = \frac{4}{15} \pi r^5 \frac{m}{V_r} \mathbf{R} \\ &= \frac{1}{5} m r^2 \mathbf{R} \end{aligned} \quad (8)$$

for a sphere, where  $V_r$  is the volume of a sphere of radius  $r$ . For an ellipsoid with radii  $a, b$  and  $c$  we get

$$\mathbf{A}_{\text{ellipsoid}} = \frac{1}{5} m \begin{bmatrix} a^2 & 0 & 0 \\ 0 & b^2 & 0 \\ 0 & 0 & c^2 \end{bmatrix} \mathbf{R} \quad (9)$$

With this extension we always get a full rank moment matrix  $\mathbf{A}$  even for a single particle! Eq. (7) can be viewed as a specialized version of GMLS [Martin et al. 2010] for the case that the deformation derivatives only contain rotations. However, in contrast to GMLS, Eq. (7) takes into consideration particles of finite size through the integral in Eq. (8) instead of single points only.

## 4 Generalized Position Based Dynamics

Before describing our overall simulation model, we explain how to evolve oriented particles in time. As an integration scheme we use PBD because shape matching is designed to work with this approach. To handle the rotational quantities we had to generalize PBD.

We will briefly recap basic position based dynamics and then explain our generalization. PBD evolves a set of particles with positions  $\mathbf{x}$  and velocities  $\mathbf{v}$  in three stages per time step. In the prediction stage a predicted position  $\mathbf{x}_p$  is computed for each particle using explicit Euler integration so  $\mathbf{x}_p \leftarrow \mathbf{x} + \mathbf{v} \Delta t$ . In the second stage, the solver corrects the predicted positions (not the velocities,

thus the name of the method) such that they satisfy a set of constraints by iterating through all constraints multiple times. Finally the particles' state variables are updated in the integration step as  $\mathbf{v} \leftarrow (\mathbf{x}_p - \mathbf{x}) / \Delta t$  and  $\mathbf{x} \leftarrow \mathbf{x}_p$ . Note that the solver's modifications on the predicted positions influence the velocities of the particles. Only this way does the resulting system become second order in time. PBD is straight forward to implement yet unconditionally stable because positional corrections never overshoot.

As in regular PBD, we define stiffness, friction and damping coefficients to be scalars  $s \in [0 \dots 1]$ . Since we use constant time steps, this definition is intuitive and works as expected. However, if the time steps are not constant, the coefficients should be defined as  $s = s' \Delta t$  to reduce time step size dependence.

### 4.1 Integration

In addition to position  $\mathbf{x}$  and velocity  $\mathbf{v}$ , our particles carry an orientation unit quaternion  $\mathbf{q}$  and the angular velocity  $\omega$ . The prediction step using explicit forward Euler integration then becomes

$$\mathbf{x}_p \leftarrow \mathbf{x} + \mathbf{v} \Delta t \quad (10)$$

$$\mathbf{q}_p \leftarrow \left[ \frac{\omega}{|\omega|} \sin\left(\frac{|\omega| \Delta t}{2}\right), \cos\left(\frac{|\omega| \Delta t}{2}\right) \right] \mathbf{q}, \quad (11)$$

For stability reasons,  $\mathbf{q}_p$  should directly be set to  $\mathbf{q}$  if  $|\omega| < \epsilon$ . After the solver has modified the predicted state  $(\mathbf{x}_p, \mathbf{q}_p)$ , as we shall describe in Section 5.1, the current state is updated using the integration scheme

$$\mathbf{v} \leftarrow (\mathbf{x}_p - \mathbf{x}) / \Delta t \quad (12)$$

$$\mathbf{x} \leftarrow \mathbf{x}_p \quad (13)$$

$$\omega \leftarrow \text{axis}(\mathbf{q}_p \mathbf{q}^{-1}) \cdot \text{angle}(\mathbf{q}_p \mathbf{q}^{-1}) / \Delta t \quad (14)$$

$$\mathbf{q} \leftarrow \mathbf{q}_p, \quad (15)$$

where  $\text{axis}()$  returns the normalized direction of a quaternion and  $\text{angle}()$  its angle. Again, for stability reasons,  $\omega$  should be set to zero directly if  $|\text{angle}(\mathbf{q}_p \mathbf{q}^{-1})| < \epsilon$ . There are two rotations,  $\mathbf{r} = \mathbf{q}_p \mathbf{q}^{-1}$  and  $-\mathbf{r}$  transforming  $\mathbf{q}$  into  $\mathbf{q}_p$ . It is important to always choose the shorter one, i.e. if  $r \cdot w < 0$  use  $-\mathbf{r}$ . As in traditional PBD for translation, changing the rotational quantity  $\mathbf{q}_p$  in the solver also affects its time derivative  $\omega$  through the integration step creating the required second order effect. Note that this formulation is a simplification of the true rigid body dynamics since it omits precession. The error introduced is zero for spherical particles and grows with the aspect ratio of the particle shape. Also, correct precession of bodies composed of multiple particles emerges automatically so the error shows only for bodies composed of a small number of particles.

### 4.2 Friction

In PBD, friction is handled by scaling down the linear velocity by a constant factor  $s$  after the update step. If a particle has collided with a solid object, we modify both, linear and angular velocities as

$$\mathbf{v} \leftarrow \mathbf{v} + (\mathbf{v}_s - \mathbf{v})_{\perp \mathbf{n}} \cdot s_{lin} \quad (16)$$

$$\omega \leftarrow \omega + \frac{\mathbf{r}}{|\mathbf{r}|^2} \times (\mathbf{v}_s - \mathbf{v} - \omega \times \mathbf{r}) \cdot s_{rot}, \quad (17)$$

where  $\mathbf{v}_s$  and  $\mathbf{n}$  are the velocity and the normal of the solid at the impact point,  $\mathbf{r} = r \mathbf{n}$  and  $r$  is the particle radius. The two scalars  $s_{lin} \in [0 \dots 1]$  and  $s_{rot} \in [0 \dots 1]$  control the amount of linear and angular friction.

The equations for handling friction in case of two particles colliding with each other look similar

$$\mathbf{v}_1 \leftarrow \mathbf{v}_1 + \left( \frac{\mathbf{v}_1 + \mathbf{v}_2}{2} - \mathbf{v}_1 \right)_{\perp \mathbf{n}} \cdot s_{lin} \quad (18)$$

$$\mathbf{v}_2 \leftarrow \mathbf{v}_2 + \left( \frac{\mathbf{v}_1 + \mathbf{v}_2}{2} - \mathbf{v}_2 \right)_{\perp \mathbf{n}} \cdot s_{lin} \quad (19)$$

and

$$\omega_1 \leftarrow \omega_1 + \frac{\mathbf{r}_1}{|\mathbf{r}_1|^2} \times (\mathbf{v}_{avg} - \mathbf{v}_1 - \omega_1 \times \mathbf{r}_1) \cdot s_{rot} \quad (20)$$

$$\omega_2 \leftarrow \omega_2 + \frac{\mathbf{r}_2}{|\mathbf{r}_2|^2} \times (\mathbf{v}_{avg} - \mathbf{v}_2 - \omega_2 \times \mathbf{r}_2) \cdot s_{rot} \quad (21)$$

where in this case  $\mathbf{n} = (\mathbf{x}_2 - \mathbf{x}_1) / |\mathbf{x}_2 - \mathbf{x}_1|$ ,  $\mathbf{r}_1 = \mathbf{r}\mathbf{n}$ ,  $\mathbf{r}_2 = -\mathbf{r}\mathbf{n}$  and  $\mathbf{v}_{avg} = (\mathbf{v}_1 + \omega_1 \times \mathbf{r}_1 + \mathbf{v}_2 + \omega_2 \times \mathbf{r}_2) / 2$ .

## 5 Simulation Model

We represent objects as a set of oriented particles and a set of edges connecting them. The resulting mesh does not need to have the topology of a triangle or tetrahedral mesh. It might look similar to a tetrahedral mesh locally in places where the model is volumetric. In other locations, where thin structures are present, it can take the form of a particle chain as in Fig. 3(d).

### 5.1 Implicit Shape Matching

This data structure is simulated by defining one shape matching group per particle. A group contains the corresponding particle and all the particles connected to it via a single edge. In sparse regions of the mesh, regular shape matching would become immediately unstable in this setting while in our case there are no limitations to the connectivity structure.

After the prediction step, the solver iterates multiple times through all shape match constraints in a Gauss-Seidel type fashion. For each constraint the goal positions are computed using Eq. (4). All the particles of the group are then moved towards their goal position by the same fraction  $s_{stiffness}$  which mimics stiffness as in [Müller et al. 2006]. This stiffness can be specified per particle as Fig. 3(c) shows.

In terms of orientation, we only update the orientation of the center particle by replacing it with the optimal rotation provided by shape matching. Generalized shape matching, as we formulated it in Section 3, has a nice property: it only influences the orientation of the particle along the directions contained in the moment matrix  $\mathbf{A}$ . Let us have a look at two extreme cases. If there is only one particle in the group, generalized shape matching will return the orientation of that particle (see Eq. (7)) so the solver does not change it as expected. If the number of particles in the group and their positions are such that they robustly span a 3D space, the new orientation of the particle is dominated by the orientation of the entire group. All situations in between smoothly interpolate these two cases. In case of a chain of particles, for instance, the orientations of the particles along the direction of the chain are determined by shape matching, while they can freely rotate about the axis along the chain.

### 5.2 Stretching vs. Bending

Shape matching per node models both stretching and bending resistance at the same time. This is sufficient in many cases. If the artist wishes to specify them separately, we support regular PBD distance constraints on the edges as well. So in order to reduce bending resistance only, the shape match stiffness can be made small and

the distance constraints activated. However, if the shape matching stiffness is set to zero, shape matching still has to be performed to update the orientations of the particles for collisions and skinning since the distance constraints only act on the positions.

### 5.3 Explicit Shape Matching

We also support additional explicit shape matching groups defined by the user which can cover arbitrary subsets of the particles in the mesh. The implicit shape matching as described above is not performed for particles in explicit groups. Their positions and orientations are controlled by the explicit group only. In contrast to implicit shape matching, all participating particles get the shape match rotation. This results in rigid components, as shown in several examples in Section 9. There is one exception: particles belonging to more than one explicit shape matching group are treated as non-oriented (i.e. the matrix  $\mathbf{A}_i$  in Eq. (7) is set to zero for particle  $i$ ). This allows us to model various joints as in the monster truck sample. Without this exception rotation information would propagate from one group to the other and prevent the wheels from rotating freely.

### 5.4 Plastic Deformation

We allow the explicit shape matching groups to deform plastically as well. Plastic deformation starts whenever one of the group particles is involved in a collision with relative velocity higher than a user defined threshold. When this happens, we simply deactivate the shape matching group for a fixed number of simulation frames (5 in our case) and let implicit shape matching take over. After the deformation phase, we reactivate the explicit group and absorb the deformation into the rest state.

Special care has to be taken when updating the rest state. It is crucial that explicit shape matching groups do not store individual rest positions of particles. The rest configuration of the model needs to be consistent over all shape matching groups otherwise ghost forces appear. Also, when distorting the rest poses, the original poses are still needed. We call these original poses the bind poses.

Let  $b_i, r_i$  and  $d_i$  be rigid transformations of the form  $f(\mathbf{x}) = \mathbf{R}\mathbf{x} + \mathbf{t}$ , where  $\mathbf{R}$  is a rotation matrix. The bind pose transformations  $b_i$  map a zero centered and axis aligned particle to its original position and orientation defined by the user and the algorithm we will describe in Section 6.2. Before plastic deformation occurs, the rest poses  $r_i$  correspond to the bind poses so  $\forall i: r_i = b_i$ . The transformations  $d_i$  map the particles from the rest pose to the current pose and, thus, describe the deformation of the object. These are the transformations that shape matching gives us. Without plastic deformation, they can directly be used for mesh skinning (see Section 7). However, as soon as the rest poses do not correspond to the bind poses anymore, the transformations to be used for skinning are  $d_i \circ r_i \circ b_i^{-1}$  (application from right to left). In contrast, the poses of the particles for collision detection are  $d_i \circ r_i$ .

After the deformation phase, particle  $i$  deviates from the rest pose  $r_i$ . This deviation is stored in the transformation  $d_i$ . We want to absorb this transformation by changing the rest pose  $r_i$ . However, since the object in world space is not aligned with the rest state during the simulation, the world space deformation  $d_i$  needs to be transformed back into the rest state space before being absorbed. Since the transformation from rest to world space varies over the object, we have to pick a specific one and choose the transformation  $s$  returned by the shape matching group at the first frame after re-activation. At this point in time we let  $r_i \leftarrow s^{-1} \circ d_i \circ r_i$ .



## 5.5 Torsion Resistance

To control torsion resistance we iterate through all the edges and update the rotations of the adjacent particles similar to friction handling as

$$\mathbf{q}_1 \leftarrow \text{slerp} \left( \mathbf{q}_1, \mathbf{q}_2, \frac{1}{2} s_{\text{torsion}} \right) \quad (22)$$

$$\mathbf{q}_2 \leftarrow \text{slerp} \left( \mathbf{q}_2, \mathbf{q}_1, \frac{1}{2} s_{\text{torsion}} \right). \quad (23)$$

The function  $\text{slerp}(\mathbf{q}_1, \mathbf{q}_2, s)$  returns  $\mathbf{q}_1$  if  $s = 0$ ,  $\mathbf{q}_2$  if  $s = 1$  and the spherical interpolation of  $\mathbf{q}_1$  and  $\mathbf{q}_2$  for values in between.

## 6 Collision Handling

In traditional PBD particles are represented by spheres. Using these spheres as collision primitives results in bumpy collision representations of objects. This is problematic because bumpy surfaces introduce unnatural friction and other visual artifacts. The fact that our particles have orientation information lets us represent them by ellipsoids which can more accurately approximate flat surfaces, as demonstrated in [Yu and Turk 2010] in the case of fluid simulations (see Fig. 3(c)). Two questions arise. First, how to handle ellipsoids correctly in the collision detection step and, second, how to find their principal radii prior to the simulation.

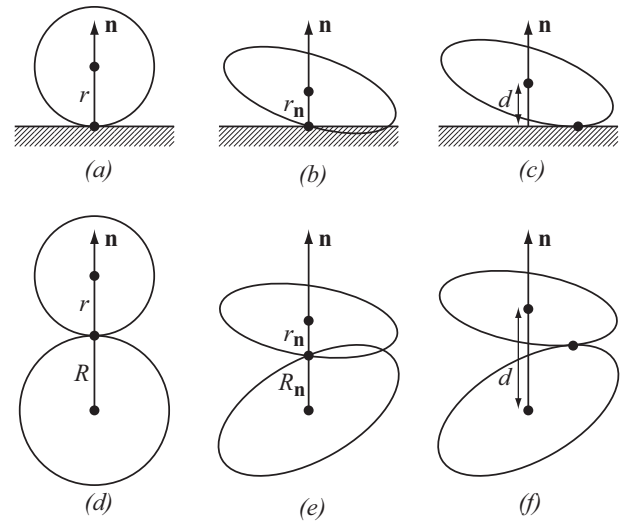
### 6.1 Ellipsoid Collision

When using spheres, a collision with a plane occurs when the particle gets closer to the plane than its radius  $r$ . In this case, PBD shifts the particle up along the plane normal  $\mathbf{n}$  such that it touches the plane as shown in Fig. 2(a). For an ellipsoid the situation gets a bit more complicated. A simple approximate way to resolve the collision is to use the ellipse's radius in the direction of  $\mathbf{n}$  for collision handling. As Fig. 2(b) shows, the collision is only resolved correctly if  $\mathbf{n}$  is aligned with the principal axes of the ellipsoid. In Appendix A.1 we show how to compute the correct distance  $d$  shown in Fig. 2(c). This is slightly more expensive. The situation is similar in the case of particle-particle collisions shown in Fig. 2(d)-(f) for which we derive the necessary equations in Appendix A.2.

### 6.2 Ellipsoid Representation of Objects

Let us assume the user has already placed the particles to simulate a given graphical mesh and specified radii for each of them. We use the same radius for all particles depending on the spacing of the particles to be able to use spatial hashing [Teschner et al. 2003] effectively for finding overlapping particles.

We now wish to automatically compute the principal axis directions and radii for each particle. Our algorithm to do this is similar to the method proposed in [Yu and Turk 2010] for creating smooth surfaces of particle fluids. For a certain particle we collect all graphical vertices within the particle's radius. We then compute the co-variance matrix of this vertex cloud relative to the particle's center and use polar decomposition to determine the orientation of the ellipsoid. In contrast to [Yu and Turk 2010] we do not use the eigenvalues of the co-variance matrix to determine the radii of the ellipsoid because the eigenvalues are related to the sum of squared distances and not the distances themselves. Instead, we use the extents of the oriented bounding box (OBB) of the mesh vertices as principal radii. For stability reasons we clamp the radii to the range  $[\frac{1}{\sigma}r \dots r]$ , where  $\sigma$  is a limit on the aspect ratio. In our examples we use  $\sigma = 2$ .



**Figure 2:** Rows: particle-plane collision, particle-particle collision. Left to right: sphere collision, approximate and correct ellipsoid collision

This approach can be extended in a useful way. Instead of adjusting only the orientations of particles, one can also move them to the center of mass of the surrounding vertices. By iterating these two steps multiple times, particles placed on the surface of the graphical mesh typically drift towards the center of a feature such as an arm. We used this property to create the skeleton shown in Fig. 3(c).

## 7 Visual Mesh Skinning

A common method to animate an arbitrary graphical mesh is to embed it in a tetrahedral simulation mesh as in [Müller and Gross 2004]. While the tetrahedral mesh deforms, each graphical vertex is transformed along with it, using barycentric weighting of the vertex positions of the closest tetrahedron. The method is only robust if each graphical vertex is surrounded by a tetrahedron because only in that case all the barycentric coordinates are positive, as noticed by [Twigg and Kacic-Alesic 2010]. Many tetrahedra are typically required to get an accurate approximation of the graphical mesh's surface, particularly if the graphical mesh has a complex structure with many branches that need to be resolved in order to move independently.

The fact that, in our case, each particle stores a full transformation composed of translation and rotation from the rest state to the current state lets us use a more robust approach for binding the graphical to the simulation particles. For each graphical vertex we precompute links and weights to nearby particles (at most 4 in our case) and use linear blend skinning to compute its transformation. This yields plausible deformations even if the visual geometry is far away from the simulated particles and for very sparse simulation meshes. Skinning remains well defined for the case even of a single simulation particle. We found that simple linear blend skinning was sufficient. However, to further improve the results, dual quaternion blending [Kavan et al. 2008] could be used as well.

## 8 Simulation Framework

We built a design and simulation framework to create various demo scenes. The design tool is used to create a physical representation of an object. After loading a graphical mesh, the user places



**Figure 4:** This underwater scene with a lion fish and six plants demonstrates simulation and collision handling of thin features.

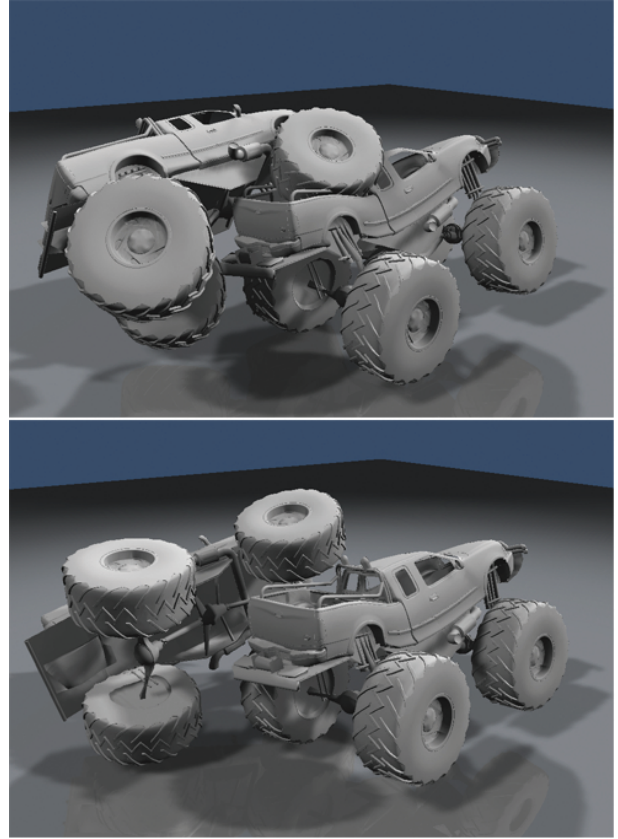
multiple simulation particles on the mesh surface with a spray tool. Alternatively, particles can also be placed individually for more precise control. Next, with an additional spray tool, edges are generated within the tool's radius between particles that are closer than a specified distance. Edges can also be defined individually. After creating the skinning weights automatically, the model can already be simulated as a preview in the editor. There are other tools to paint node and torsion stiffnesses; to define explicit shape matching groups; to adjust skinning weights; and to automatically fit ellipsoids.

While creating the physical model only takes a few minutes, painting the skinning weights is often tedious. The difficult part is to make sure vertices are not linked to particles belonging to other independently moving parts of the object like another tentacle of the octopus shown in Fig. 3. There is a simple way to fix this problem in our tool. The user first selects a subset of the simulation particles for instance all the particles on one tentacle. Then the user selects a subset of the graphical vertices. The skinning tool then creates links for the selected vertices only and binds them only to selected simulation particles. This step is repeated until all vertices are linked.

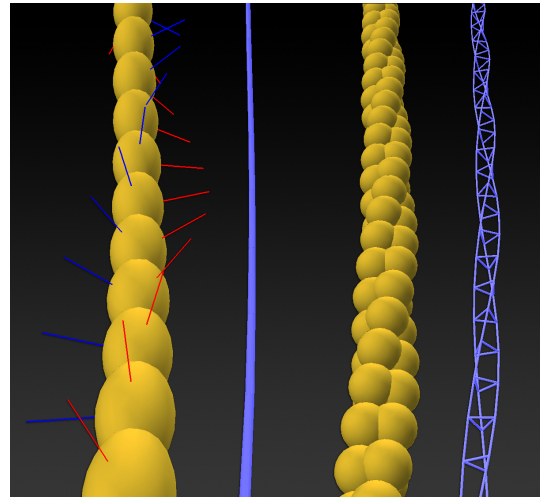
## 9 Results

All our examples were executed in real time on a single core of an Intel Core2 CPU at 2.4 GHz and an NVIDIA Quadro FX 5800 GPU. Although we are working on a GPU version of the solver we list CPU figures here to demonstrate that the method is fast even if run sequentially.

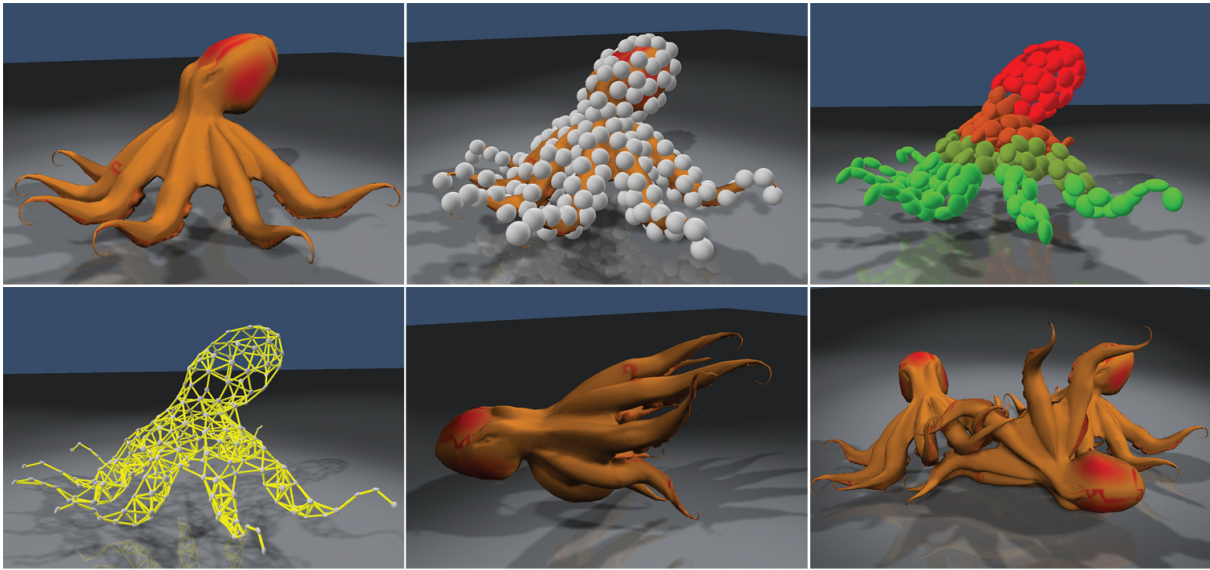
Fig. 3 shows how a physical model is created from a graphical mesh using our framework. First, nodes were painted using the spray tool with a specified distribution density. The auto ellipsoid tool generated the orientations and shapes of the ellipsoids automatically. The user has painted per particle stiffness parameters shown in colors from 0 (green) to 1 (red). The edge spray tool was then used to create edges with a certain maximal length. As Fig. 3(d) shows, the tips of the tentacles are one dimensional particle chains. Near the head the mesh has the form of a triangulated surface, while further down it fills space and looks more like a tetrahedral mesh. The simulation mesh consists of 300 particles and 750 edges, while the visual mesh has 5k vertices and 10k triangles. Simulating this model including self collision handling takes 4ms, and skinning and rendering 6ms. For three instances interacting with each other as shown in Fig. 3(f) the solver takes 10ms.



**Figure 5:** Two monster trucks colliding. The lower car's autobody deforms plastically during the crash.



**Figure 6:** Left: A rope represented by a chain of ellipsoids with orientation information. Right: Six times more non oriented spherical particles are needed to capture the twist dynamics and to get a collision volume of comparable accuracy.



**Figure 3:** Model creation. From top left to bottom right: (a) Original graphical mesh, (b) particles created with spray tool, (c) ellipsoid fit and stiffness painted, (d) edges created with spray tool, (e) soft deformation, (f) complex collisions and interaction.

Fig. 4 shows an underwater scene with a lion fish and six plants. Buoyancy forces are applied to plant nodes. As initial condition we added random impulses to the plant leaves to break visual similarity in the first frame. Only the fish's head motion is scripted. All other dynamic effects are a results of the simulation. There are a total of 1k particles with 3.5k edges while the the visual mesh has a total of 30k vertices and 48k triangles. The solver and skinning take 10ms and 9ms, respectively.

Since our method is conceptually most closely related to the elaston approach [Martin et al. 2010], we created this scene to compare our method to their lion fish sample. The authors report a total computational cost of 7s per frame. We are, thus, about three orders of magnitude faster with more visual degrees of freedom on the fish and interaction with complex plants. From a pure computation point of view, this comparison is not fair since we only use 1k particles in total versus 5k elastons. However, for computer graphics applications, it is the visual result that is most relevant. If that can be achieved with a much simpler model, the direct comparison is justified.

The monster truck visual mesh shown in Fig. 1 and Fig. 5 is composed of 63k vertices and 100k triangles. The model we use to simulate it contains only 300 particles and 800 edges. Due to this setup, skinning is the bottleneck in these scenes. Simulating the two interacting trucks in Fig. 5 and skinning the meshes takes 8ms and 30ms respectively. With rendering, the scene still runs at the interactive rate of 20 fps. We achieved the effect of free spinning wheels by creating three explicit shape matching groups, one for the autobody and one for each axle. Since the autobody group shares exactly two particles with each of the axle groups, there remains one degree of freedom for each axle too rotate freely.

The advantage of our approach in terms of model and time complexity over traditional shape matching and the SPH-based approach of [Becker et al. 2009] is depicted in Fig. 6. On the left a rope is represented by a chain of ellipsoidal particles with orientation. The edge structure is the simple string shown in blue. To capture the twist dynamics and to get a collision volume of comparable accuracy, six times more regular spherical particles are needed. Again, the edge structure is shown in blue. This time each

cross section is composed of three connected particles such that the orientation of the rope can be represented by the locations of the particles alone. To avoid gaps in the collision volume the number of segments has to be doubled yielding six times more particles and twelve times more edges. Since the running time is dominated by per particle shape matching and the cost of integrating orientation information is negligible, our method is about six times faster then traditional shape matching in this scenario.

## 10 Conclusion and Future Work

We have presented a novel method based on shape matching to simulate various types of solid using only a small number of particles connected by meshes with arbitrary topology. The key idea to make shape matching work in sparse regions of the mesh is to add orientation and rotation information to the particles. This information can also be used to create more accurate collision volumes as well as skinning graphical meshes to the physical model. Our content creation editor enables artists to create versatile physical models in a matter of minutes from any visual mesh.

Being a geometric approach, our model has its disadvantages as well. It is hard to make the simulation time step independent if variable time steps are required. Also, the physical behavior is dependent on the mesh geometry and does not converge to a certain solution with mesh refinement as models based on continuum mechanics. This is not a big issue unless one would want to auto-generate models of various resolutions for run time level of detail. This is an interesting problem for further research.

The two main bottlenecks are currently shape matching and skinning of the visual mesh. We are in the process of writing a parallel, GPU accelerated version of our solver. Also, having focussed on solids only so far, a next step would be to study the interaction of solids with fluids in a unified framework.

## Acknowledgements

We thank NVIDIA for supporting our research and the PhysX team, especially Gordon Yeoman, for the valuable inputs and feedback.



## References

- BARAFF, D., AND WITKIN, A. 1998. Large steps in cloth simulation. *Proceedings of ACM Siggraph*, 43–54.
- BARGTEIL, A. W., WOJTAN, C., HODGINS, J. K., AND TURK, G. 2007. A finite element method for animating large viscoplastic flow. *ACM Transactions on Graphics* 26, 3 (July), 16:1–16:8.
- BECKER, M., IHMSEN, M., AND TESCHNER, M. 2009. Corotated sph for deformable solids. In *Eurographics Workshop on Natural Phenomena*, 27–34.
- BERGOU, M., AUDOLY, B., VOUGA, E., WARDETZKY, M., AND GRINSUN, E. 2010. Discrete Viscous Threads. *SIGGRAPH (ACM Transactions on Graphics)*.
- BERTAILS, F., AUDOLY, B., CANI, M.-P., QUERLEUX, B., LEROY, F., AND LÉVÊQUE, J.-L. 2006. Super-helices for predicting the dynamics of natural hair. *ACM Transaction on Graphics* 25, 3 (July), 1180–1187.
- BERTAILS, F. 2009. Linear time super-helices. *Computer Graphics Forum* 28, 2 (Apr.), 417–426.
- BRIDSON, R., MARINO, S., AND FEDKIW, R. 2003. Simulation of clothing with folds and wrinkles. In *ACM SIGGRAPH Symposium on Computer Animation*, 28–36.
- GERSZEWSKI, D., BHATTACHARYA, H., AND BARGTEIL, A. W. 2009. A point-based method for animating elastoplastic solids. In *Proceedings of the 2009 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, ACM, New York, NY, USA, SCA '09, 133–138.
- GOLDENTHAL, R., HARMON, D., FATTAL, R., BERCOVIER, M., AND GRINSUN, E. 2007. Efficient Simulation of Inextensible Cloth. *SIGGRAPH (ACM Transactions on Graphics)* 26, 3.
- GRINSUN, E., HIRANI, A. N., DESBRUN, M., AND SCHRÖDER, P. 2003. Discrete shells. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation*, Eurographics Association, Aire-la-Ville, Switzerland, Switzerland, SCA '03, 62–67.
- JANSSON, J., AND VERGEEST, J. S. M. 2003. Combining deformable- and rigid-body mechanics simulation. In *In The Visual Computer*, SpringerVerlag, 280–290.
- KAVAN, L., COLLINS, S., ZARA, J., AND O'SULLIVAN, C. 2008. Geometric skinning with approximate dual quaternion blending. ACM Press, New York, NY, USA, vol. 27, 105.
- LENOIR, J., AND FONTENEAU, S. 2004. Mixing deformable and rigid-body mechanics simulation. In *Proceedings of the Computer Graphics International*, IEEE Computer Society, Washington, DC, USA, 327–334.
- MARTIN, S., KAUFMANN, P., BOTSCH, M., WICKE, M., AND GROSS, M. 2008. Polyhedral finite elements using harmonic basis functions. *Computer Graphics Forum* 27, 5, 1521–1529.
- MARTIN, S., KAUFMANN, P., BOTSCH, M., GRINSUN, E., AND GROSS, M. 2010. Unified simulation of elastic rods, shells, and solids. *ACM Trans. on Graphics (Proc. SIGGRAPH)* 29, 3, 39:1–39:10.
- MÜLLER, M., AND GROSS, M. H. 2004. Interactive virtual materials. In *Graphics Interface 2004*, 239–246.
- MÜLLER, M., KEISER, R., NEALEN, A., PAULY, M., GROSS, M., AND ALEXA, M. 2004. Point based animation of elastic, plastic and melting objects. In *the ACM SIGGRAPH 2004 Symposium on Computer Animation*, 141–151.
- MÜLLER, M., HEIDELBERGER, B., AND TESCHNER, M. 2005. Meshless deformations based on shape matching. In *Proc. SIGGRAPH 2005*, 471–478.
- MÜLLER, M., HENNIX, B. H. M., AND RATCLIFF, J. 2006. Position based dynamics. *Proceedings of Virtual Reality Interactions and Physical Simulations*, 71–80.
- O'BRIEN, J. F., AND HODGINS, J. K. 1999. Graphical modeling and animation of brittle fracture. In *Computer Graphics (SIGGRAPH '99 Proceedings)*, ACM Press, New York, 137–146.
- O'BRIEN, J. F., ZORDAN, V. B., AND HODGINS, J. K. 1997. Combining active and passive simulations for secondary motion. In *Proceedings of SIGGRAPH 1997, Technical Sketch*.
- O'BRIEN, J. F., BARGTEIL, A. W., AND HODGINS, J. K. 2002. Graphical modeling and animation of ductile fracture. In *Computer Graphics (SIGGRAPH 2002 Proceedings)*, 291–294.
- PAI, D. K. 2002. STRANDS: Interactive simulation of thin solids using Cosserat models. *Computer Graphics Forum* 21, 3 (Sept.), 347–352.
- PAULY, M., KEISER, R., ADAMS, B., DUTRÉ, P., GROSS, M., AND GUIBAS, L. J. 2005. Meshless animation of fracturing solids. *ACM Trans. Graph.* 24 (July), 957–964.
- PROVOT, X. 1995. Deformation constraints in a mass-spring model to describe rigid cloth behavior. *Proceedings of Graphics Interface*, 147–154.
- RIVERS, A. R., AND JAMES, D. L. 2007. Fastlsm: Fast lattice shape matching for robust real-time deformation. In *ACM Transactions on Graphics (Proc. SIGGRAPH 2007)*, vol. 26(3), 82:1–82:6.
- SCHMEDDING, R., AND TESCHNER, M. 2008. Inversion handling for stable deformable modeling. In *The Visual Computer*, vol. 24, 625–633.
- SIFAKIS, E., SHINAR, T., IRVING, G., AND FEDKIW, R. 2007. Hybrid simulation of deformable solids. In *Proceedings of the 2007 Symposium on Computer Animation*, 81–90.
- SPILLMANN, J., AND TESCHNER, M. 2007. CORDE: Cosserat rod elements for the dynamic simulation of one-dimensional elastic objects. In *Proceedings of the 2007 Symposium on Computer Animation*, Eurographics Association, 63–72.
- STAM, J. 2009. Nucleus: Towards a uni?ed dynamics solver for computer graphics. In *In IEEE International Conference on Computer-Aided Design and Computer Graphics*, 1–11.
- SZELISKI, R., AND TONNESEN, D. 1992. Surface modeling with oriented particle systems. *SIGGRAPH Comput. Graph.* 26 (July), 185–194.
- TESCHNER, M., HEIDELBERGER, B., MÜLLER, M., POMERANERTS, D., AND GROSS, M. 2003. Optimized spatial hashing for collision detection of deformable objects. *Proc. Vision, Modeling, Visualization VMV 2003*, 47–54.
- TWIGG, C., AND KACIC-ALESIC, Z. 2010. Point cloud glue: Constraining simulations using the procrustes transform. *ACM SIGGRAPH/Eurographics Symposium on Computer Animation*.
- VOLINO, P., MAGNENAT-THALMANN, N., AND FAURE, F. 2009. A simple approach to nonlinear tensile stiffness for accurate

cloth simulation. *ACM Trans. Graph.* 28 (September), 105:1–105:16.

WOJTAN, C., AND TURK, G. 2008. Fast viscoelastic behavior with thin features. *ACM Transactions on Graphics* 27, 3 (Aug.), 47:1–47:8.

YU, J., AND TURK, G. 2010. Reconstructing surfaces of particle-based fluids using anisotropic kernels. *ACM SIGGRAPH/Eurographics Symposium on Computer Animation*.

## A Appendix

### A.1 Ellipsoid-Plane Collision

Given an ellipsoid centered at the origin with principal radii  $a, b, c$  and orientation  $\mathbf{R}$  and a plane  $p$  as  $\mathbf{n}^T \mathbf{x} = d$ . To resolve a potential collision we first compute the contact point which is the point on the ellipsoid with normal parallel to  $\mathbf{n}$ , i.e. to which  $p$  is tangent. The ellipsoid can be defined as the zero iso-surface of

$$c(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x} - 1, \quad (24)$$

where

$$\mathbf{A} = \mathbf{R} \begin{bmatrix} \frac{1}{a^2} & 0 & 0 \\ 0 & \frac{1}{b^2} & 0 \\ 0 & 0 & \frac{1}{c^2} \end{bmatrix} \mathbf{R}^T \text{ and } \mathbf{A}^{-1} = \mathbf{R} \begin{bmatrix} a^2 & 0 & 0 \\ 0 & b^2 & 0 \\ 0 & 0 & c^2 \end{bmatrix} \mathbf{R}^T. \quad (25)$$

The point  $\mathbf{x}$  we are looking for meets the two constraints

$$\nabla c(\mathbf{x}) = \lambda \mathbf{n} \text{ and} \quad (26)$$

$$c(\mathbf{x}) = 0 \quad (27)$$

With  $\nabla c(\mathbf{x}) = 2\mathbf{A}\mathbf{x}$  we solve the first constraint for  $\mathbf{x}$  yielding  $\mathbf{x} = \frac{1}{2}\mathbf{A}^{-1}\lambda\mathbf{n}$ . Substituting into the second constraint and solving for  $\lambda$  gives

$$\lambda = \pm \frac{2}{\sqrt{\mathbf{n}^T \mathbf{A}^{-1} \mathbf{n}}} \text{ and so } \mathbf{x} = \pm \frac{\mathbf{A}^{-1} \mathbf{n}}{\sqrt{\mathbf{n}^T \mathbf{A}^{-1} \mathbf{n}}}. \quad (28)$$

The solution for which  $\mathbf{n}^T \mathbf{x}$  is smaller is the contact point. If this point is below the plane, i.e.  $\mathbf{n}^T \mathbf{x} < d$  there is a collision.

### A.2 Ellipsoid-Ellipsoid Collision

Let us assume we have two ellipsoids, both with their center at the origin and we want to separate them by moving ellipsoid 2 along the direction defined by the normal vector  $\mathbf{n}$ . For the two ellipsoids we have the constraint functions

$$c_1(\mathbf{x}) = \mathbf{x}^T \mathbf{A}_1 \mathbf{x} - 1 \text{ and} \quad (29)$$

$$c_2(\mathbf{x}) = (\mathbf{x} - d\mathbf{n})^T \mathbf{A}_2 (\mathbf{x} - d\mathbf{n}) - 1. \quad (30)$$

We are looking for two scalars  $d$  and  $\lambda$  and a contact point  $\mathbf{x}$  such that

$$c_1(\mathbf{x}) = 0 \quad (31)$$

$$c_2(\mathbf{x}) = 0 \quad (32)$$

$$\nabla c_1(\mathbf{x}) = \lambda \nabla c_2(\mathbf{x}). \quad (33)$$

The scalar  $d$  will tell us how far to shift ellipsoid 2 from the origin along  $\mathbf{n}$  such that the two ellipsoids touch (from outside if  $\lambda < 0$ ). Expanding the third constraint yields

$$\mathbf{A}_1 \mathbf{x} = \lambda \mathbf{A}_2 (\mathbf{x} - d\mathbf{n}). \quad (34)$$

Solving for  $\mathbf{x}$  gives

$$\mathbf{x} = (\lambda \mathbf{A}_2 - \mathbf{A}_1)^{-1} \mathbf{A}_2 \lambda d \mathbf{n}. \quad (35)$$

Setting  $\mathbf{B} = (\lambda \mathbf{A}_2 - \mathbf{A}_1)^{-1} \mathbf{A}_2$ , noting that the  $\mathbf{A}_i$  are symmetric and substituting into the first and second constraint results in two equations for  $\lambda$  and  $d$ :

$$\lambda^2 \mathbf{n}^T \mathbf{B}^T \mathbf{A}_1 \mathbf{B} \mathbf{n} = \frac{1}{d^2} \quad (36)$$

$$\mathbf{n}^T (\lambda \mathbf{B} - \mathbf{I})^T \mathbf{A}_2 (\lambda \mathbf{B} - \mathbf{I}) \mathbf{n} = \frac{1}{d^2} \quad (37)$$

Now we equate the two left hand sides and multiply with  $(\mathbf{B}^T)^{-1}$  from the left and with  $\mathbf{B}^{-1}$  from the right on both sides resulting in

$$\lambda^2 \mathbf{n}^T \mathbf{A}_1 \mathbf{n} = \mathbf{n}^T (\lambda \mathbf{I} - \mathbf{B}^{-1})^T \mathbf{A}_2 (\lambda \mathbf{I} - \mathbf{B}^{-1}) \mathbf{n} \quad (38)$$

Back substituting  $\mathbf{B}$ , the right hand side becomes

$$\mathbf{n}^T \left[ \lambda \mathbf{I} - \mathbf{A}_2^{-1} (\lambda \mathbf{A}_2 - \mathbf{A}_1) \right]^T \mathbf{A}_2 \left[ \lambda \mathbf{I} - \mathbf{A}_2^{-1} (\lambda \mathbf{A}_2 - \mathbf{A}_1) \right] \mathbf{n}. \quad (39)$$

Most of the terms cancel out and we are left with the very simple expression

$$\lambda = \pm \sqrt{\mathbf{n}^T \mathbf{A}_1 \mathbf{A}_2^{-1} \mathbf{n}}. \quad (40)$$

Once  $\lambda$  is known Eq. (36) can be used to compute  $d$  and Eq. (35) to evaluate  $\mathbf{x}$ . In terms of signs, we want  $\lambda$  to be negative (ellipsoids touching from the outside) and  $d$  to be positive (shifting ellipsoid 2 in the positive direction of  $\mathbf{n}$ ).

